Computer Graphics: Lecture 11

Cameras and Perspective

Welcome back!

Last time we learned about culling. We needed it to draw a solid object correctly without z-buffering.

But it didn't work when we wanted to draw two objects, so we needed to learn how to attach a z-buffer to our pipeline and render pass.

There is a particular significance to it: we need the Z-buffer if we are to be able to freely explore our scene and know that objects will be rendered correctly.

This time

This time, we are going to learn about scenes: arrangements of multiple models that share the same coordinate space.

Scenes are finally where we can start drawing “levels” or “worlds”.

We will also learn how cameras work. They do more than just draw things in perspective. They also let us choose which part of the scene to view.

Finally, we’ll learn about projections. This is where we add perspective to what we see in the camera to get that 3D appearance.

Here is today’s reading: Chapter 1.9.

A review of coordinate systems

Before we understand cameras, we need to review the idea of **coordinate systems**.

A **coordinate system** is what gives meaning to a set of coordinates.

For example, you and I could agree that the point (1, 0, 0) refers to my right shoudler. If we also agree that a unit represents a foot of distance, then maybe (-1, 0, 0) would refer to my left shoulder. (depending on the axes)

In 3D graphics, we use several coordinate systems simultaneously, and we use matrices to convert between them.

Clipping coordinates and NDCs

Just to remind you, your video card expects all the triangles that appear on screen to be at least partially within the clipping volume.

The clipping volume is x = [-w, w], y = [-w, w], z = [0, w], w <> 0.

After clipping coordinates, the w-divide occurs, meaning that a visible x and y are in [−1, 1], and a visible z is in [0, 1]. These coordinates are called **Normalized Device Coordinates (NDCs)**

The GPU draws whatever is in this cuboid. It doesn't know anything about your scene, about distance, or about units. It just draws everything inside a cubic shape.

Making coordinates meaningful

We have several different coordinate systems where we do different things. These are all meaningful to humans.

When we're done, after multiplying all our matrices out, we must end up in clipping coordinates. That signals that we are done.

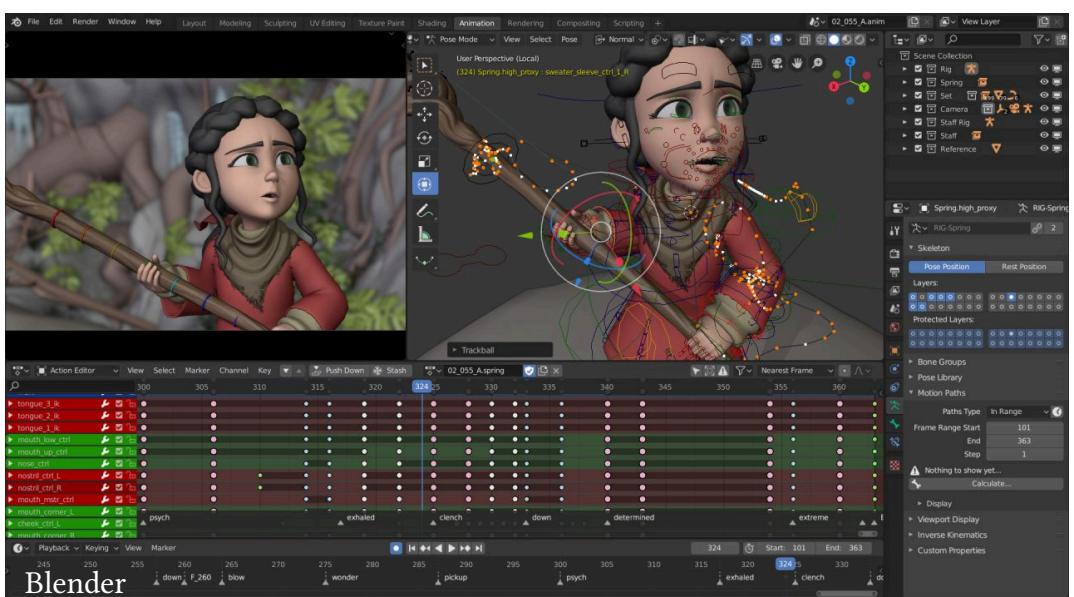
That is, our goal is to transform all the vertices in our scene so that the visible ones end up in the little box. Then the GPU will draw them.

So far, we've been putting our triangles in clipping coordinates directly. That ends now.

Model coordinates

The first coordinates we encounter are model coordinates.

These are the coordinates that describe the vertex positions of the 3D model inside the editor, *before* we arrange it with any other models.



Model coordinates (2)

Here, (0, 0, 0) usually refers to the center of the model in the 3D modelling program.

This works great if we're drawing *one thing*. We can draw it centered in the little clipping volume.

But model coordinates don't make sense globally.

Different models may use different units. For example, a car model may have it that a unit of 1 represents a meter, while a toaster might have a unit represent a centimeter.

Model coordinates (3)

If we put them next to each other without adjustment, the toaster will tower above the car.

Or more likely: the car will be *inside* the toaster, because (0, 0, 0) is usually the center of the model.

Model coordinates assume that the model is center of the world, so two different models cannot coexist without manually adjusting the coordinates and combining it into the same space.

We need some kind of adjustment to each model. To shrink or expand it, to move it into position, and to rotate it to face the way we want.

The scene

One of the fundamental units of making any kind of 3D app is the scene.

A scene is a collection of 3D objects that coexist.

When we make any kind of 3D application, we think in terms of scenes:

- Scenes are levels or maps in a game
- Scenes are scenarios in a simulation
- Scenes are literal scenes in a 3D movie

Within a scene, coordinates have meaning as distances. For example, we might assign a distance of 1 meter to a dimension's coordinate. Now, we know that an object at (0, 0, 0) and (1, 0, 0) are 1 meter apart.

Scene coordinates

One of the things we've been doing recently is moving model coordinates into **scene coordinates** (also called **world coordinates**).

This is the job of the model matrix. It's purpose is to be multiplied by every vertex in the model to move it into scene or world coordinates.

Most commonly, we will apply a scale to make the model be the right size, some rotation to have it face the way we want, and a translation to put it in the right spot. All these operations are combined into 1 matrix.

This is what we did in the last sample. We drew two cubes, but we changed the model matrix between each one to make one cube small and behind the other.

The camera

Scenes do not have any kind of inherent clipping volume. Do we just draw the whole scene? And if so, from where?

Therefore, we need a camera.

A camera describes a location, a direction, and a volume. Where to draw the scene from, which way to point, and how much of it to draw.

The camera's coordinates

A camera has a position within the scene, like anything else in it.

We use the camera's position to define a new coordinate space: called **view coordinates**.

In world coordinates, (0, 0, 0) referred to the center of the world, whatever that was (we can put it wherever we want).

In **view coordinates**, (0, 0, 0) refers to the exact position of the camera.

Therefore, we want to transform all the coordinates in the scene, to be relative to the camera. If their coordinates are close to (0, 0, 0), we want them to be physically close when we draw them.

The camera's coordinates (2)

But we don't just care about how far they are from the camera. We also want to know if they are in-view.

That means we want the x and y coordinates to be based on the center of the camera's view, and for the z coordinate to mean how far it is from the camera depth-wise.

Notice: we're starting to get closer to normalized device coordinates. Once we're in **view coordinates**, (0, 0) x and y refer to the center of the screen, just like in NDCs.

Converting to the camera's coordinates

Just like how we had a model matrix that converted our model coordinates into world coordinates, there's a matrix that will convert world coordinates into camera coordinates.

It's called the **view matrix**.

It's going to be a major focus of the lecture, but first, there's one more thing we need to discuss.

Perspective

Here is a wild fact that is shocking to learn:

- Things that are farther away appear smaller

Yet despite this shocking fact, we haven't been doing this.

When we drew the small box behind the big box, we had to manually scale it down. But that scaled the whole box. The farther vertices should have been smaller, and the edges should have been pointing towards the horizon.

You might think that we do perspective with a matrix multiplication, and we sort of do, but not in the way you think…

Doing perspective

Perspective is a non-linear transformation. But in homogenous coordinates, it's not.

All we do is change the w coordinate to represent the distance from the camera.

The matrix that does this also does other things (adjusting for field of view, and maybe for eye distance in VR), but the main thing is just changing w to be distance from camera relative to how far away we want to be able to see.

The effect of this is that farther away things will be "squished", and fit into the little clipping box.

All the transformations

So, for every model we want to draw, this is all the hoops we have to jump through to get it in the right coordinates to draw:

- It starts in model coordinates. These are the coordinates in the vertex buffer after we define them or load them from a file.
- Next, we convert to scene coordinates with the model matrix.
- Then, we convert to camera coordinates with the view matrix.
- Finally, we apply perspective and field of view (projection).

So, in the vertex shader:

```
clip_coords = projection * view * model * model_coords
```

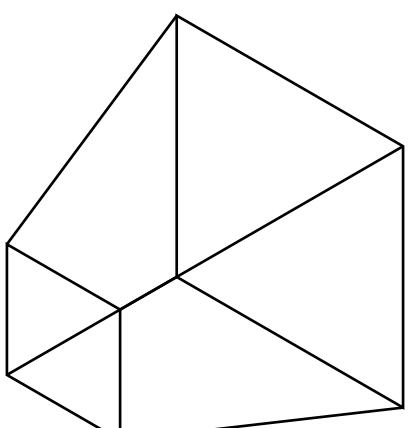
Questions?

Camera visualization

Before we go too deep, let's do some basic visualization exercises.

A camera's viewspace can be visualized using a **frustum**[1]

A frustum is basically a pyramid with its point chopped off.



[1]Note, this word only has one 'r', but people will often slip in a second 'r' and say "frustrum". It's kind of like "Sherbet".

Frustums

The near plane of the frustum is the small one.

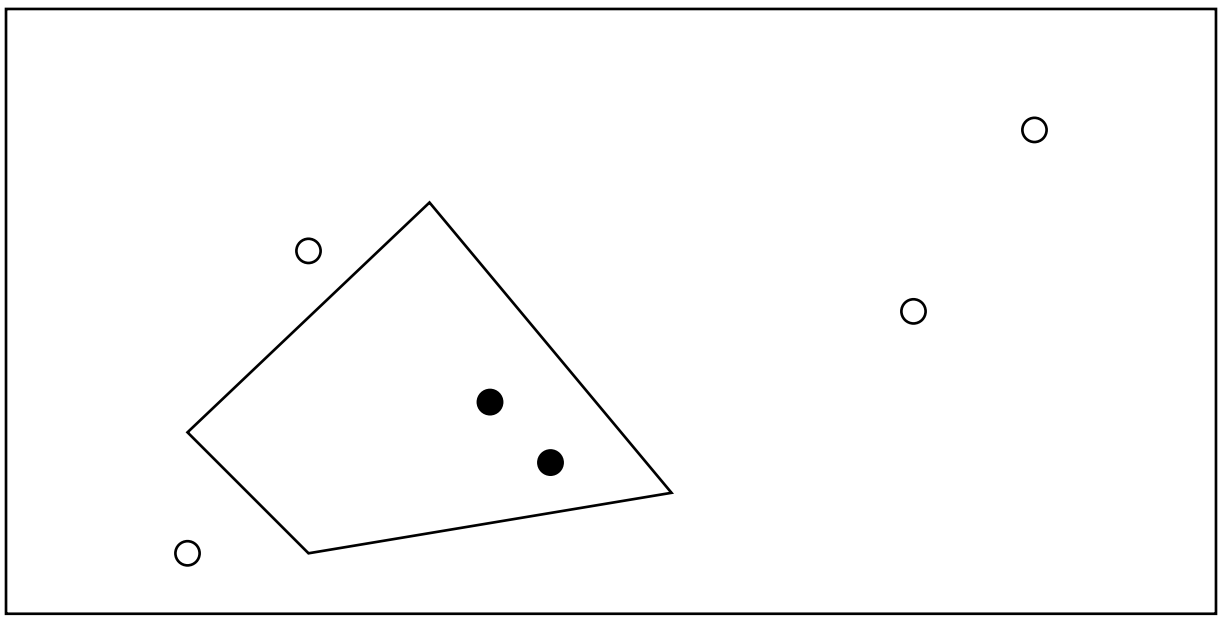
The far plane is the big one.

Then there are the left, right, top, and bottom planes.

Let's imagine that we're viewing a scene from the top down. The circles are some kind of object we might want to draw.

Everything that is visible will be inside the frustum…

Scene visualization

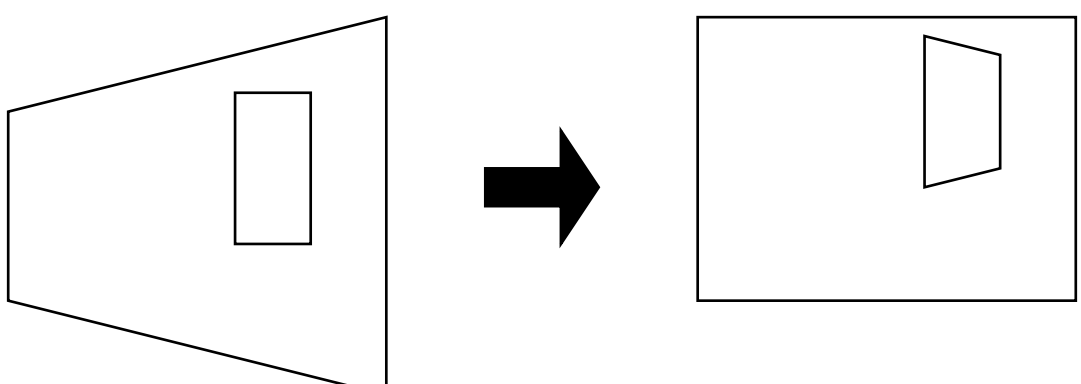


A scene from the top down. The circles are objects in the scene. Only the filled ones will be visible. The trapezoid is the camera.

The reason for the frustum

The main reason we use a frustum to represent what the camera can see is that there is more room for objects to fit in frame if they are far away.

The projection matrix will *squash* the ends of the frustum so that it forms a box. This box is the clipping volume that will be rendered.



The math involved

We already know that we can move things around in our scene by using the model matrix. Now we need two more transformations:

1. We need to arrange the coordinates so that they refer to how "in frame" they are and how far away from the camera they are (**view**)
2. We need to determine the bounds of the frustum and do the squashing at the ends (**projection**)

Let's start with view.

Coordinate switch

Remember how, when learning about depth buffers, we used left-handed coordinates? That is, +Z went *into* the screen, because that's what the depth buffer expected?

Now we're going to use right-handed coordinates. +Z goes *out of* the screen.

Why? Because `gl-matrix` just assumes you want to use right-handed coordinates. It will invert the Z direction for you.

Therefore, in view coordinates, something with a Z of −20 means that it is 20 units away from the camera in the direction it's facing. If Z is +20, it's behind the camera.

View math

Suppose the camera is at position (1, 1, 1) world coordinates

And suppose an object is at position (5, 1, 1), world.

Suppose the camera is facing right (in the +X direction).

What should the object's camera coordinates be?

To answer this quesiton, think about it like this:

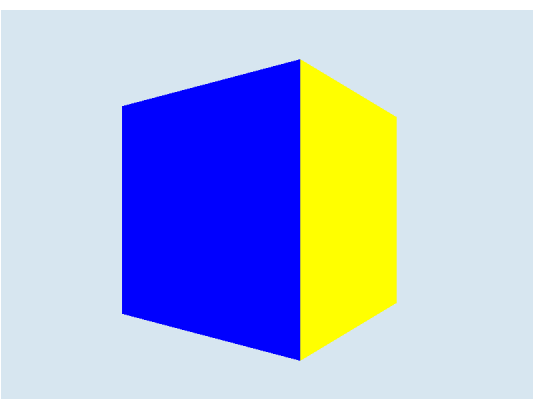
- Will the object be visible at all?
- If so, where in frame will it be? That is its new X,Y
- How far away from the camera will it be? That is its new -Z coordinate (using right handed coordinates).

View math (2)

The answer is, in view coordinates, the object is at (0, 0, −4)

It is directly in the center of the frame, which is why X and Y are (0, 0)

And it 4 units in front of the camera, which is where the −4 comes from.



I drew the above cube at (5, 1, 1), with a camera at (1, 1, 1). It's centered.

View math (3)

Okay, suppose again the camera is at position (1, 1, 1) in world coords.

Suppose now it is facing in the +Z direction.

Where would the object need to be in world coordinates to be directly centered, and 10 units away from the camera?

View math (4)

Answer: since the camera is facing in +Z, imagine a line in the +Z direction, 10 units long.

In fact, that vector will take us from the camera to the object.

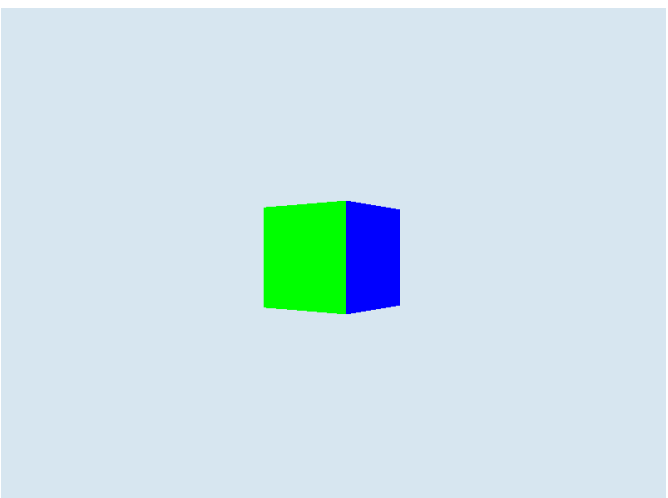
So (1, 1, 1) + (0, 0, 10) = (1, 1, 11) world coordinates

In camera coordinates, it would be (0, 0, −10). It's centered in the frame, but its 10 units ahead.

If the camera moved to (0, 0, 0), the same object would now be a little to the upper left of the center of the screen.

View math centered but farther

Here's what it looks like to draw an object at (1, 1, 11) with a camera at (1, 1, 1), looking toward +Z.



Notice that it's smaller. That's because it's farther away.

View math (5)

Now, let's say we draw an object 100 units away. We've decided that 100 units is near the edge of what we can see, so the object is pretty small.

The camera is facing the -Z direction.

We move the camera *forward* in the -Z direction.

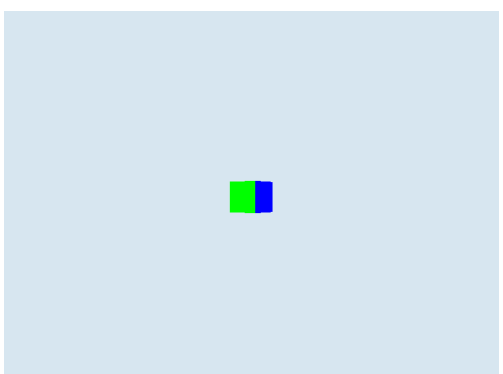
If the object is the only thing in the scene, is there any difference at all between the camera moving in the -Z direction, and the object moving in the +Z direction?

View math (6)

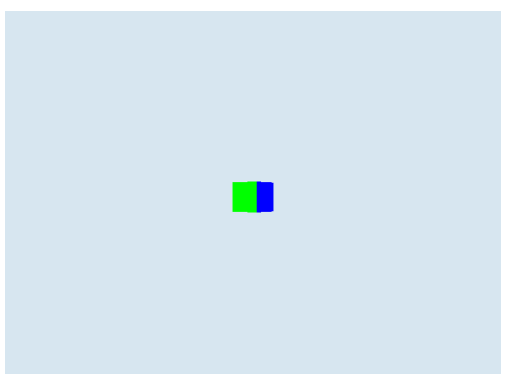
No, actually. They are the same. We can't actually mathematically separate the two, because the resulting clipping coordinates will be identical.

Fundamentally, the camera translating in one direction is the same as everything else in the scene translating in the opposite direction.

object at position 100



object at position 25



object at 100, camera at 75

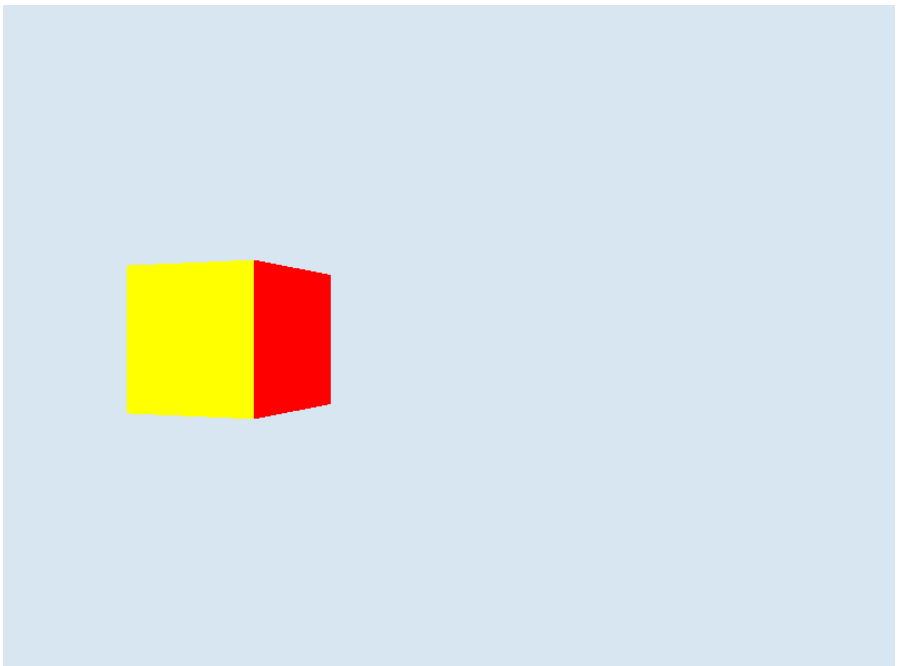
View math (7)

Suppose the camera is looking right at the object.

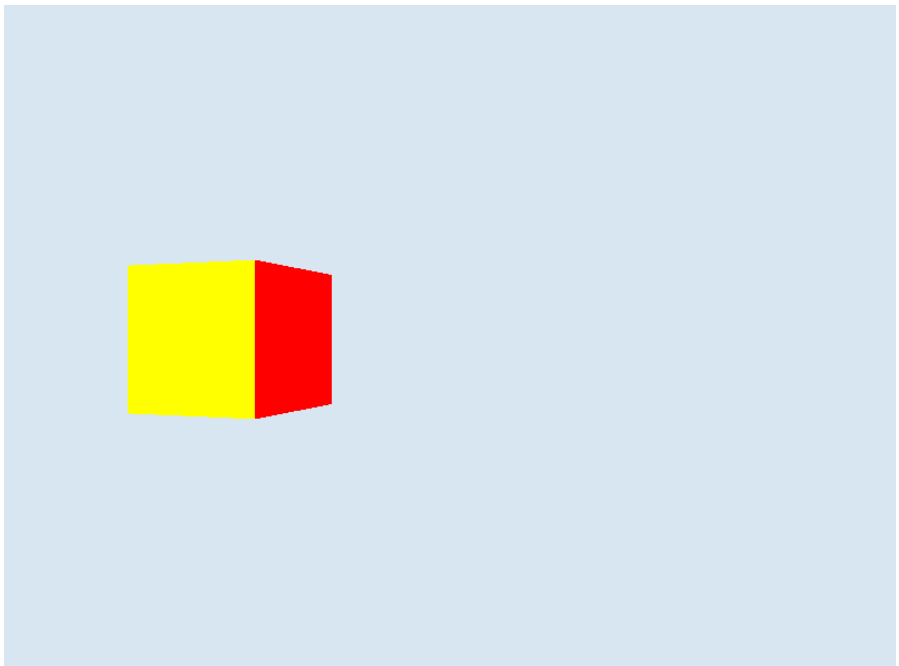
The camera rotates a bit to the right, causing the object to slide a bit to the left.

Is there any difference between this, and just rotating the object to the left (around the camera's origin, not in place)?

Camera rotation illustration



Rotated the camera right $5\%\tau$.



Rotated the object $-5\%\tau$ about 0

Can you tell a difference? I can't.

View math (8)

There is a special relationship between the camera's motions and the motions of the objects that we draw: they are inverses.

Specifically, the view matrix is the **inverse** of the model matrix for the camera.

Wait, the camera has a model matrix?

Yes. Every object that is positioned in the scene has a model matrix.

But the view matrix is the inverse of the camera's model matrix.

Camera matrices

To summarize, every object in our scene will have its own model matrix.

The model matrix will determine the position, rotation, and scale of any given object. The camera is no exception.

However, when it comes time to render, we will *invert* this matrix for the camera.

This inverse matrix is called the **view matrix**. It will take vertices from world coordinates into view coordinates.

Camera matrices (2)

Why? Because inverting the view matrix will make the camera's position the origin, and the camera's direction the inverse of the axes.

This will frame object coordinates in terms of camera position/ orientation.

We multiply every world vertex by this view matrix.

We obtain world vertices by multiplying by the model matrix.

Therefore, so far, we have `clip_pos = view * model * position`

Moving the camera around

Suppose we want the camera to be at point (2, 1, 1), and pointing in the X+ direction.

How would we do that?

Moving the camera around

We translate the camera's model matrix, and rotate it:

```
const cam = mat4.create();
mat4.translate(cam, cam, vec4.fromValues(2, 1, 1))
mat4.rotateY(cam, cam, -.25 * TAU);
// (the camera starts out looking in the -z direction,
// we'll learn why next time, but that's why we rotate
// right instead of left)
```

At this point the camera is just another object in the scene, like a cube that we want to rotate.

Generating the view matrix

However, the closer the camera gets to something, the closer that thing gets to the screen.

And the more the camera rotates left, the more everything else rotates right.

We want the effect on the models in the scene to be the *inverse* of the camera matrix. This inverse is called the view matrix:

```
const view = mat4.create();
mat4.invert(view, cam)
```

Using the view matrix

Once we have this matrix, we can send it to the shader to be multiplied by our vertices in world coordinates.

We want to wait though, we also need a projection, and we're about to learn perspective.

One useful shortcut, however. It's so common to need to move the matrix and have it look at a point, that there's a built-in operation that does that in our matrix library...

Using the view matrix (2)

```
mat4.lookAt(view,
  vec3.fromValues(0, 0, 0), // camera position
  vec3.fromValues(0, 0, -1), // what point to look at?
  vec3.fromValues(0, 1, 0)); // which direction is up?
```

This function outputs an already-inverted view matrix you can use.

If you want the *model* matrix (pre-inverted), you can use `mat4.targetTo`, which takes the same parameters but returns an uninverted matrix.

Questions?

Perspective

That explains how view coordinates work. But here's a secret: I've been using perspective to generate all those camera screenshots.

If I didn't, everything would be the same size regardless of how far away it was.

So, how do we actualy do the perspective calculation?

That is, what is the relationship between how big something is, and how far away it is?

Perspective math

The simple answer is:

$$x' = \frac{x}{d}$$

$$y' = \frac{y}{d}$$

That is, the perspective adjusted x value is equal to the camera's x value divided by the distance. Same for y.

We basically just divide by distance.

If something is twice as far away, its x and y values are twice as close to the center of the screen.

Perspective math (2)

The problem is that the distance of each vertex is different for each vertex. If it were a constant we could just divide everything by it.

But we need to do this in a single matrix multiplication.

Now's where that w-divide comes in handy. This is the whole point of it. It's also called "perspective divide".

All we do is use the z-value to compute the w-value.

Then, the GPU divides x, y, and z, by the w-value. Achieving perspective.

The simplest projection matrix

Here's a really simple matrix that does that: $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \end{pmatrix}$

It keeps the x and y values the same, but throws away the z values (because we're projecting onto a flat plane) and uses the old z values as new w values. (negating them to convert handedness)

After the w-divide, the x and y values are correct.

This is simple, but…it doesn't quite work for us.

Problems with the simplest projection matrix

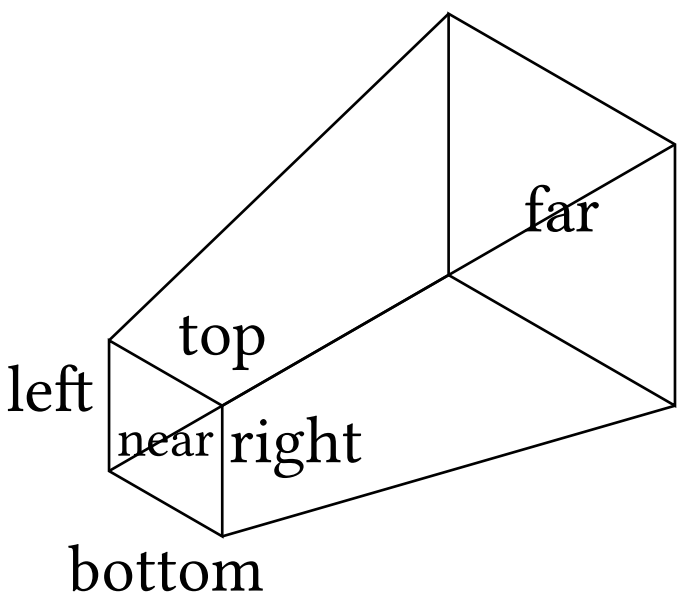
There are a few problems with it:

1. We don't want to throw the z-value away. We want to keep it so that it can be written into the depth buffer.
2. We want configurability. How far is "far"? We want to be able to define the range at which the object is right up in our face, and how far away it can get before disappearing from view. This matrix makes it so that z can vary between 0 and 1, but what if we want z to vary between 1 (nearest) and 1000 (farthest)?
3. It assumes that the screen is square. What if we want to configure the field of view? Then we need to widen or shrink the frustum in either direction.

Understanding a projection matrix

A projection matrix can be defined by the positions of its 6 planes.

Specifically: near, far, top, right, left, and bottom.



Understanding a projection matrix (2)

By specifying a number for each of those planes, we can construct a matrix that performs the correct “smooshing” on any vector in it.

The goal is: for x and y, project values inside onto the near plane.
for z: scale the value to efficiently use the range [0, 1]
for w: scale so that w becomes the relative distance between near and far. (1 for near, so we don’t scale).

But what do the numbers represent?

Understanding a projection matrix (3)

The near and far numbers are distances from the camera's origin.

We can think of the camera's origin as roughly equivalent to the eye (in fact, in VR, it is exactly the eye. Tracking the camera's origin to the eye for both eyes independenly is what gives a VR effect).

The near plane value is the distance from the origin to the screen.

This value is where the camera "starts". Anything closer will get clipped, and we'll see "into" it.

Likewise, the far value is the plane at which objects disappear. It should be far enough away that objects don't obviously blink out of view.

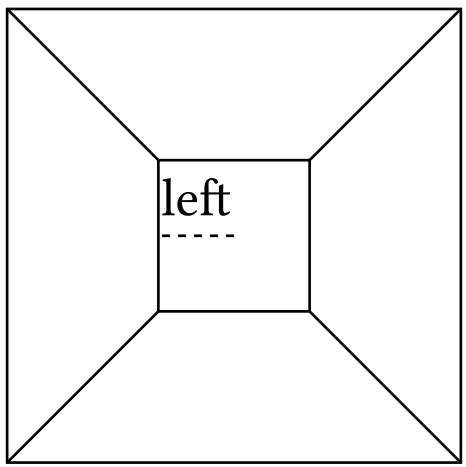
Understanding a projection matrix (4)

Typically, it's nice to use distance units that are meaningful and consistent. For example, if your scene is of a normal human scale, it makes sense to use "meters" as the distances. Then the near plane could be, e.g., 0.25 meters away, and the far plane could be 2 kilometers away.[1]

The top, bottom, left, and right values describe the distance from the middle of the near frame to the nearest edge of the plane:

[1] you might be tempted to make the near plane super close, like 0.00001 meters away, to reduce clipping. Unfortunately, every time you halve the near plane value or double the far plane value, you lose a bit of precision in your depth buffer. We'll talk about why that matters.

Understanding a projection matrix (5)



The frustum from the front, showing the left plane value, which is the distance from the center of the near plane to its left edge. Increasing the left and right values will make the near plane wider.

Our goal

First, let's figure out the values for those 6 planes.

The near plane is the first stop: pick a value that's acceptable. How close should the camera be able to get before we start clipping into things.

Most games or sims will use a collision system to prevent the camera from getting too close. It's often quite challenging when it's a free camera.

Remember: you can't make it too close.

For simplicity, let's make it 1. Anything closer than 1 will be invisible.

Z-fighting

Now for the far plane. How far away do we want to be able to see?

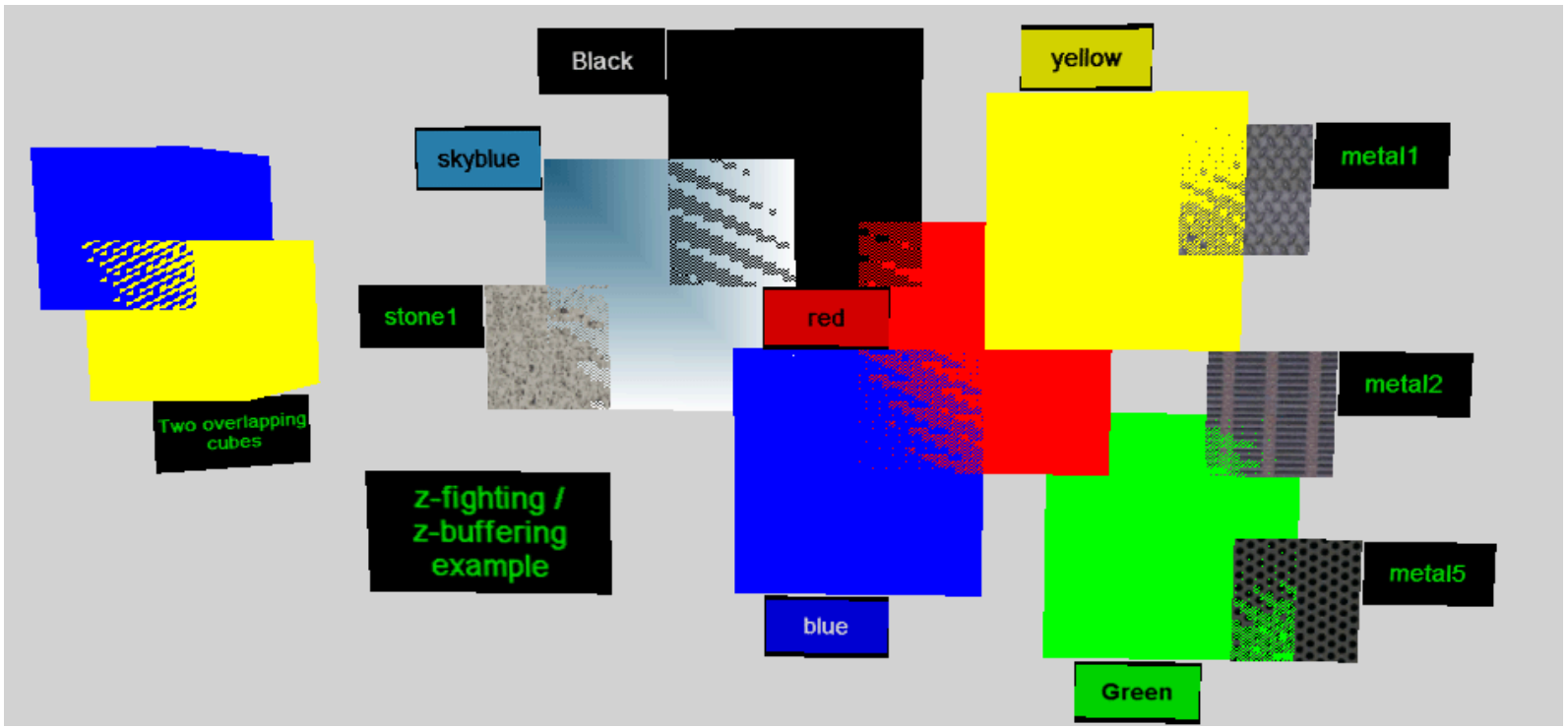
You might think “I want to see everything”, but there’s a limit.

The depth buffer contains values between 0 and 1, and it will typically have 24 or 32 bits of precision.

The wider the distance is between near and far, the more bits you need.

Halving the near plane distance uses a bit, and so does doubling the far plane distance. What happens if we use too many?

Z-fighting (2)



Z-fighting is what happens when two triangles overlap in the Z-buffer.

(Image by CompuHacker, Public Domain)

Choosing plane values

Some applications might need huge view distances (e.g., space sims). For those, you might need to break the rendering up into multiple passes with different frustums.

But suppose we choose reasonable near and far values. What about top, bottom, left, and right?

Fundamentally, the relationship between them comes from the aspect ratio:

$$\text{AspectR} = \frac{\text{width}}{\text{height}} = \frac{r-l}{t-b}$$

Aspect ratio and Field of View

The aspect ratio is just the ratio between width and height.

For my samples, I've been using an 800-by-600 canvas, which is a 4:3 ratio, or 1.3333 if expressed as a decimal.

Then, there's **field of view (FOV)**, which is how wide the view angle is.

Most people use *horizontal field of view* expressed in degrees to determine how wide the frustum is. A typical PC game will have a horizontal field of view of between 60 and 85 degrees. Console games have narrower fields of view, typically.[1]

[1]Sitting far from the screen makes a narrow field of view more appropriate. Small FOV can reduce the cost to render indoor environments.

Aspect ratio and Field of View (2)

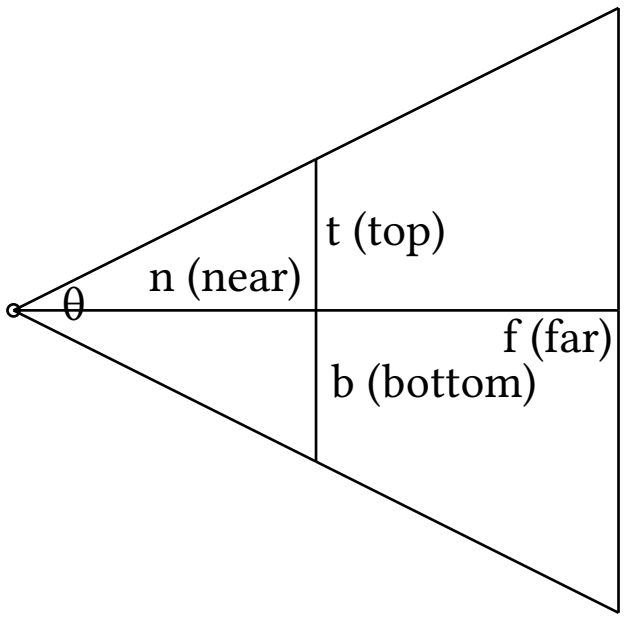
For historical reasons, in 3D programming APIs, **vertical field of view (vFOV)** is more commonly used, including in gl-matrix.

To get the vFOV, we just take the horizontal FOV and divide by the aspect ratio. So if we want 60 degrees to be the FOV (1/6 τ) and we have a 4:3 canvas, we divide 60 degrees by 4/3, which is 45 degrees (1/8 τ).

Let's say we pick that. Now what are our top and bottom planes?

Let's draw a triangle…

Determining `t` and `b`



A side view of the frustum. The angle between the middle and the top is θ / 2. $\tan\left(\frac{\theta}{2}\right) = \frac{t}{n} \rightarrow n \cdot \tan\left(\frac{\theta}{2}\right) = t$

Determining l and r

Once we've solved for t and b, getting (l - r) distance just means multiplying the (t - b) distance by the aspect ratio.

And once we have the (l - r) distance, we can get all the individual values.

t = (t - b) / 2. b = -t. r = (r - l) / 2. l = -r.

We've solved for all the plane offsets. But what matrix does that correspond to?

Solving for the projection matrix

Let's remind ourselves of the simplest projection matrix:

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

We have two things that will change how x and y will behave on the near plane…

Solving for the projection matrix (2)

On the one hand, if the distance from the eye to the near plane increases, everything visible gets bigger because it's closer to the screen. The screen has width 2 (from 1 to −1). So we scale by 2n. The 2 offsets the fact that (r - l) is twice the distance from the center to the edge.

The denominator is (r - l), which is the total distance which is being mapped to the coordinate '1'. Top and bottom is similar.

$$\begin{pmatrix} \left(\frac{2n}{r-l}\right) & 0 & 0 & 0 \\ 0 & \left(\frac{2n}{t-b}\right) & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

If we're doing VR...

This isn't going to be an issue for this class, but it's interesting, so I thought I'd mention it.

In VR, each eye has a slightly different position. We can translate the eye by adding a constant term to x and y. We do this for both eyes, so we get two slightly different projection matrices.

Remember the w-divide. We have to multiply the amount of translation by z, so that when w takes on the value of z, we divide it out again:

$$\begin{pmatrix} \left(\frac{2n}{r-l}\right) & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \left(\frac{2n}{t-b}\right) & \frac{t+b}{t-b} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Linearly mapping depth values

There's one more critical thing to do. Right now, our matrix throws away the z-values. They always become zero.

We can't have that. The GPU will do the w-divide, and then set the depth buffer equal to the resulting z value.

So, we need to convert the z value to a value between 0 in 1, where 0 means touching the near plane, and 1 means touching the far plane. Anything outside that should get clipped.

It seems like a simple linear map could work: $z' = \frac{z-n}{f-n}$

However, after the w divide happens, this will not be in the right range.

Linearly mapping depth values (2)

If we truly wanted to do this, we could.

After performing the linear mapping, but *before* returning from the vertex shader, we could multiply `position.z *= position.w`.

Then, after the w-divide, we'd get the z value back.

This works fine, but there's an approach that is much more common.

This approach allows us to do everything in the matrix, and has a nice side effect: we get more depth precision for closer objects than for further ones.

Non-linearly mapping depth values (2)

Let's try to find some values to plug into the matrix that will give us the [0, 1] depth mapping of Z.

Recall that the near plane is at position -n, and far at -f.

The new z coordinate, which I will call z' ("z-prime"), was produced from some linear function: $z' = Az + B$

The new w coordinate, was set to be the negative of the old z coord.

Therefore, after the w-divide:

$z' = \frac{z}{w'} = \frac{Az+B}{-z} = -A - \frac{B}{z}$

Non-linearly mapping depth values (3)

We want that when $z = -n \rightarrow z' = 0$,

And when $z = -f \rightarrow z' = 1$

So, plugging into the equation on the previous slide:

- $0 = -A - \frac{B}{-n}; A = \frac{B}{n}; An = B$
- $1 = -A - \frac{B}{-f}; A = \frac{B}{f} - 1$

Then, substitute $An = B$

- $A = \frac{An}{f} - 1; A - \frac{An}{f} = -1; Af - An = -f \rightarrow A = \frac{f}{n-f}$

Finally, substitute again to solve for B:

- $An = B; A = \frac{B}{n}; \frac{B}{n} = \frac{f}{n-f}; B = \frac{nf}{n-f}$

What did that do?

Remember, after the w-divide, $z' = \frac{z}{w'} = \frac{Az+B}{-z}$

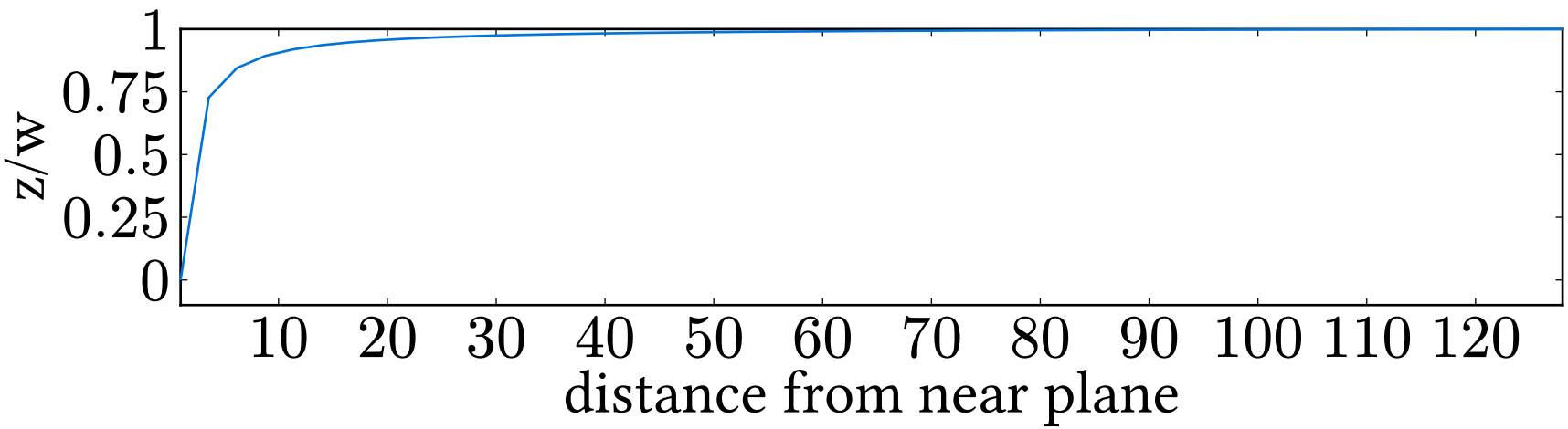
The purpose was to make is to that the z value, after the w divide, has the correct value for the z-buffer. And we just solved for A and B.

Therefore, we can now modify our matrix to have these values. We put 'A' in the 'z' column, because it gets multiplied by z. We put B in the 'w' column, because it's just a constant addition (w is 1 for input points):

$$\begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & A & B \\ 0 & 0 & -1 & 0 \end{pmatrix} = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & \frac{f}{n-f} & \frac{nf}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Those z-coordinates

Notice: the resulting z values will be nonlinear. Specifically, they will follow a plot that looks like this (using near = 1, far = 128):



Notice how most of the precision is from 1 to 10. That is desirable. We still have plenty for farther values, even if it doesn't look like it

Questions?

The perspective matrix

So that's the perspective matrix. We choose our near and far, pick an aspect ratio and field of view, and then solve for top, bottom, left, and right…

or we let our matrix library do it for us:

```
mat4.perspectiveZO( // NO is for z = [-1, 1], ZO's for [0, 1]
  viewProj, 0.15 * TAU, // output and FOVy
  context.canvas.width / context.canvas.height, // aspect R
  1, 128); // near and far
```

But we always want to understand every line of code we write, so we had to make sure we *could* generate one of these.

What is `viewProj`?

In the previous slide, we saved the perspective matrix to a variable called `viewProj`.

This is the projection matrix multiplied by the view matrix.

You see, we always multiply projection by view:

```
vo.pos = proj * view * model * position;
```

However, `proj` and `view` will be the same for every vertex in the object. We don't move the camera while we're drawing something.

What is `viewProj`? (2)

Therefore, it makes sense to combine `proj` and `view` by pre-multiplying them: `viewProj = proj * view`

```
// in typescript:
const proj = mat4.perspectiveZO(...);
const camera = ... // move the camera where you want it
const view = mat4.create(); mat4.invert(view, camera);
const viewProj = mat4.create();
mat4.mul(viewProj, proj, view); // this goes in a bindgroup
```

Now we only have one matrix to multiply in the shader, which is faster because it gets multiplied by every single vertex in the scene.

Multiple bind groups

In fact, we normally only change camera positions once per frame (maybe a couple of times if we're doing advanced lighting calculations such as shadow mapping).

For that reason, we can set the `viewProj` matrix and just leave it alone. We don't need to change it for every object we draw like we did for the `model` matrix.

Therefore, we like to put `viewProj` in a different bind group. That lets us change per-object things, like the `model` matrix, without changing per pass variables such as `viewProj`.

Homework

Be sure to review `sample08`.

It demonstrates setting up a camera, using perspective, and putting a little cube into position.

There's also a spinning cube elsewhere in the scene. Try to move the camera so that both cubes are visible!

Read the book chapter. Be aware though, that there's an error in it: the author thinks that NDC values for z need to be in the range [−1 ,1], but this is only for OpenGL. For WebGPU, the range is [0, 1]. Therefore, you'll notice the perspective matrix they use is not the same as mine.

Questions?